

Date: 20-09-2026

Lab 04 : TO UNDERSTAND AND OBSERVE THE RUNNING TIME OF TWO SIMPLE SORTING ALGORITHMS, INSERTION SORT AND MERGE SORT

Objective:

- Understand and apply basic sorting algorithms.

Introduction:

Sorting means arranging data in a particular order, like ascending or descending. Sorted data is easier to search, analyze, and manage. In this lab, we study Insertion Sort and Merge Sort, two algorithms that solve the same problem in very different ways, and we observe how their behavior and running time change with the size of the data.

Abstract:

This lab explores Insertion Sort and Merge Sort through Python implementations. Both programs take an array of integers from the user, sort it in ascending order, and display the original and sorted arrays. The running time of each algorithm was observed to understand how an incremental approach (Insertion Sort) compares with a divide-and-conquer approach (Merge Sort).

Procedure:

1. Launched Google Colab and opened a new notebook.
2. Accepted a list of integers from the user as input.
3. Wrote the Insertion Sort function, which takes one element at a time and places it in its proper position among the already sorted elements.
4. Wrote the Merge Sort function, which repeatedly divides the list into halves and then merges the sorted halves.
5. Printed the original array for each program.
6. Executed both programs and printed the sorted arrays.
7. Compared the outputs and confirmed that both produced the correct ascending order.

Theory

Date: 20-09-2026

Insertion Sort:

Insertion Sort works by growing a sorted section of the list one element at a time. It takes the next unsorted element and moves it leftward into the correct spot in the sorted section, pushing larger elements one step to the right. The idea is the same as sorting a hand of playing cards as you pick them up. The algorithm is very fast when the data is small or almost sorted, but becomes slow when the data is large and disordered. Its time complexity is $O(n^2)$ in the average and worst cases and $O(n)$ in the best case, and it needs only $O(1)$ extra space.

Write a program to implement **Insertion Sort**. The program should take an array of integers as input and sort the elements in **ascending order**. Display the original and sorted arrays.

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and key < arr[j]:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key  
    return arr  
  
user_input = input("Enter integers separated by spaces: ")  
arr = list(map(int, user_input.split()))  
  
print("Original array:", arr)  
sorted_arr = insertion_sort(arr.copy())  
print("Sorted array:", sorted_arr)  
  
... Enter integers separated by spaces: 48 39 15 99 62  
Original array: [48, 39, 15, 99, 62]  
Sorted array: [15, 39, 48, 62, 99]
```

Date: 20-09-2026

Merge Sort:

Merge Sort follows the divide-and-conquer strategy. The list is repeatedly split into two halves until each piece has a single element, which is already sorted by definition. These small pieces are then merged in pairs, always picking the smaller front element, until the full sorted list is rebuilt. Since the list is halved about $\log n$ times and each level of merging takes linear time, its time complexity is $O(n \log n)$ in all cases. This makes it well suited for large datasets, although it requires $O(n)$ additional memory for merging.

Write a program to implement **Merge Sort** using the divide-and-conquer approach. The program should take an array of integers as input and sort the elements in **ascending order**. Display the original and sorted arrays.

```
def merge_sort(arr):  
    if len(arr) > 1:  
        mid = len(arr) // 2  
        left_half = arr[:mid]  
        right_half = arr[mid:]  
  
        merge_sort(left_half)  
        merge_sort(right_half)  
  
        i = j = k = 0  
        while i < len(left_half) and j < len(right_half):  
            if left_half[i] < right_half[j]:  
                arr[k] = left_half[i]  
                i += 1  
            else:  
                arr[k] = right_half[j]  
                j += 1  
            k += 1  
  
        while i < len(left_half):  
            arr[k] = left_half[i]
```

Date: 20-09-2026

```
        i += 1
        k += 1

    while j < len(right_half):
        arr[k] = right_half[j]
        j += 1
        k += 1
    return arr

user_input = input("Enter integers separated by spaces: ")
arr = list(map(int, user_input.split()))

print("Original array:", arr)
sorted_arr = merge_sort(arr.copy())
print("Sorted array:", sorted_arr)
```

Enter integers separated by spaces: 107 99 74 8 10
Original array: [107, 99, 74, 8, 10]
Sorted array: [8, 10, 74, 99, 107]

Conclusion:

This lab demonstrated two different approaches to sorting. Insertion Sort is simple and efficient for small or nearly sorted data, but its quadratic time makes it unsuitable for large inputs. Merge Sort is more complex and uses extra memory, yet it scales far better because of its $O(n \log n)$ complexity. From this, we learned that the choice of sorting algorithm should depend on the size and nature of the data being sorted.